

Introduction to USD

- [What is USD?](#)
- [Why use USD?](#)
- [What can USD do?](#)
 - [USD can represent:](#)
 - [USD can compose and override:](#)
 - [USD/Hydra can image:](#)
 - [USD can be extended/customized:](#)
- [What can't USD do?](#)
 - [No GUIDS](#)
 - [Not a rigging system](#)
- [Heritage of USD at Pixar](#)

What is USD?

Pipelines capable of producing computer graphics films and games typically generate, store, and transmit large quantities of 3D data, which we call “scene description”. Each of many cooperating applications in the pipeline (modeling, shading, animation, lighting, fx, rendering) typically has its own special form of scene description tailored to the specific needs and workflows of the application, which is neither readable nor editable by any other application. **Universal Scene Description (USD)** is the first publicly available software that addresses the need to robustly and scalably interchange and augment arbitrary 3D scenes that may be *composed* from many elemental assets.

USD provides for interchange of elemental assets (e.g. models) or animations. But unlike other interchange packages, USD also enables assembly and organization of any number of assets into virtual sets, scenes, shots, and worlds, transmitting them from application to application, and non-destructively editing them (as *overrides*), with a single, consistent API, in a single scenegraph. USD provides a [rich toolset](#) for reading, writing, editing, and rapidly previewing 3D geometry, shading, lighting, physics, and a growing number of other graphics-related domains. In addition, because USD’s core scenegraph and [composition engine](#) are agnostic of any particular domain, USD can be extended in a maintainable way to encode and compose data in other domains.

Concretely, USD is an [open source project](#) released under the [TOST license](#).

! See also

[USD Basics in an Hour\(ish\)](#) for a quick intro to USD.

Why use USD?

USD is the core of Pixar's 3D graphics pipeline, used in every 3D authoring and rendering application, including Pixar's proprietary *Presto* animation system. Pixar is deeply committed to evolving and improving USD to address the following ongoing production concerns:

- **Provide a rich, common language for defining, packaging, assembling, and editing 3D data, facilitating the use of multiple digital content creation applications.**

Like many other interchange packages, USD provides a low-level data model that stipulates, at a "file format level", how data is encoded and organized, plus a (extensible) set of high-level schemas that provide meaningful API's and organization for concepts like [a mesh](#) or [a transform](#). With such a foundation one can create asset definitions with geometric, material, lighting, and other properties. But USD goes further to provide a freely combinable set of [Composition Arcs](#) that can be used to package, aggregate, vary, and override primitive elements and assets, with a high-performance runtime evaluation engine, embodied in a compact scenegraph known as a [Stage](#), for resolving the resulting [composed scene description](#) and extracting (and authoring) data from it.

- **Allow multiple artists to collaborate on the same assets and scenes.**

USD's most basic composition arc, [the subLayers operator](#), facilitates multiple artists in different departments, or within the same department, to simultaneously work on the same asset or scene, by allowing each artist to work in their own file (called a [Layer](#)), all of which will be combined and resolved in a [strength ordering](#) clearly specified in the USD files themselves. This ability is not a magic bullet that can automatically adjust shading data in a stronger layer when the modeling artist changes the topology of geometry defined in a weaker layer, but it allows each artist to work independently without erasing or editing any other artist's work, and helps to provide a clear audit trail of changes that aids in addressing problems like the changing-topology problem.

- **Maximize artistic iteration by minimizing latency.**

As in many media, one of the most important ingredients to achieving high-quality digital art is the ability to iterate quickly and often on a design, an asset, an animation. One of the most prominent impediments to iteration in 3D art is the speed with which an artist can get "good enough" visual feedback on the results of their edits, and the speed with which they can migrate new data between multiple applications, or restore a session that has crashed. Speed is a primary, ongoing goal of the USD project at Pixar; we continue to explore algorithmic improvements, better ways to leverage modern multi-core systems and GPU's, and compression techniques to minimize latency in accessing remotely stored data.

If your needs are similar to or are a subset of the above, then USD may be an attractive choice.

What can USD do?

USD can represent:

USD organizes data into hierarchical namespaces of [Prims](#) (short for “primitive”). In addition to child prims, each prim can contain [Attributes](#) and [Relationships](#), collectively known as [Properties](#). Attributes have [typed values](#) that can vary over time; Relationships are multi-target “pointers” to other objects in a hierarchy, and USD takes care of remapping the targets automatically when referencing causes namespaces to change. Both prims and properties can also have (non-time-varying) metadata. Prims and their contents are organized into a file abstraction known as a [Layer](#).

Built on top of this low-level, generic scene description, USD provides a set of schemas that establish a standard encoding and client API for common 3D computer graphics concepts like:

Geometry

The [UsdGeom schemas](#) define [OpenSubdiv](#) -compliant meshes, transforms, curves, points, nurbs patches, and several intrinsic solids. It also defines: the concept of arbitrary [primvars](#) as attributes that can interpolate across a geometric surface; geometric extents and aggregate, computed bounding boxes; [pruning visibility](#); and an attribute called [Purpose](#) that expresses a (non-animatable) conditional visibility useful for deploying level-of-detail proxies and guides.

Shading

The [UsdShade schemas](#) define primitive shader nodes that can be connected into networks and packaged into reusable materials, on which one can create a public interface of attributes that will drive parameters in the contained shader networks. UsdShade also provides flexible mechanisms for *binding* geometry to materials so as to define their lighting response (and physics) characteristics.

Model and Asset

USD’s composition operators allow you to construct arbitrarily large, complex scenes. As an aid to processing, analyzing, and decomposing such scenes, USD formalizes the concepts of [Model](#) and [Asset](#). The “model” prim classification allows scenegraphs to be partitioned into logical, manageable chunks for traversal, working-set management, and data coalescing/caching. The concept of “asset” shows up in USD at two levels: as a core datatype for referring unambiguously to an external file, identifying which data needs to participate in [asset/path resolution](#); and in the [AssetInfo](#) schema for depositing a record of what assets have been referenced into a scene, which survives even if the scene is [flattened](#).

USD can compose and override:

The following is a very compact description of USD's composition semantics, with links to more detailed descriptions.

You can “stack” USD layers together using the [subLayers composition arc](#), and the composition engine will resolve the data contained in such ordered (nestable) “[LayerStacks](#)” similarly to how layers in Photoshop are composed.

Any prim in a layer can also contain one or more [references composition arcs](#) that target a prim in another (or the same!) layer, and composes the tree rooted at the target prim into the referencing prim - this is the primary way to assemble elemental assets into aggregates and complete scenes. The [payload arc](#) provides a “deferred reference” that can be selectively “loaded” (or unloaded) from a [Stage](#) after the stage was initially opened; judicious use of payloads allows you to structure scenes so that clients can easily manage “working sets”, keeping in memory just the parts of the scene they need for the task at hand.

[VariantSets](#) allow an asset *creator* to bundle different multiple variations of an asset all into a single package with a “variant selector” that downstream asset *consumers* can switch, non-destructively, in stronger layers to change the variation they desire; any prim can define multiple VariantSets, which can vary along dependent or independent axes.

The last two composition arcs, [inherits](#) and [specializes](#) both establish a persistent (across further, upstream composition arcs) relationship between a “base” prim and a “derived” prim, such that the derived prim receives all of the overrides applied to the base prim anywhere in the composition; the technical difference between [inherits](#) and [specializes](#) lies in the particulars of when derived's opinions “win out” over base's opinions, but practically the difference is: you can use [inherits](#) to easily “mass edit” all instances of a particular class of prim or asset, and you can use [specializes](#) to create a “derived” that is always a “specialized” refinement of “base” in all views of your scene.

The most powerful and unifying aspect of USD's composition semantics is that all of the above operators can be applied to any prim, in any combination, and the composition engine will [resolve the resulting graph in a predictable way](#). The other desirable property that falls out of this uniform treatment of composition arcs is that stronger layers in a composition can override the scene description in weaker layers *uniformly*, regardless of whether the weaker layers were subLayered, referenced, inherited, etc. A stronger layer can override the following with respect to weaker layers:

- **Add new prims** including entire subtrees rooted at the added prim
- **Deactivate** prims, which is USD's method for non-destructive (and reversible) prim/subtree deletion
- **Reorder prims**, since in some contexts, the namespace-ordering can be meaningful
- **Add or remove Variants** to an existing VariantSet
- **Add or remove entire VariantSets**, or targets to inherit or specialize
- **Override the value of schema and user-level metadata** on a prim or property
- **Add new properties** to a prim
- **Reorder properties** on a prim. If not explicitly ordered, properties are enumerated in dictionary order
- **Override the value of any attribute** (an override value blocks all weaker timeSamples/splines)
- **Block the value** of an attribute, so that it will appear to have no authored value (or no authored animated value, if using an **animation block**)
- **Add, remove, and reorder targets on a relationship or attribute connection**

Finally, USD provides a handful of scenegraph-level features that can greatly expand the types and scale of datasets encodable in USD. The two most prominent are **native prim Instancing** for very compactly encoding (and processing) large numbers of instances/copies of a referenced asset or prim, applicable when the copies do not need to be deeply edited; and **Value Clips**, which allow timeSamples for a set of prims to be distributed across many files, and (re-)sequenced and retimed non-destructively.

USD/Hydra can image:

Hydra is the imaging framework that ships as part of the USD distribution. It connects “scene delegates” (that consume scene data) and “render delegates” (that send the scene data to particular renderers), in such a way that render and scene delegates can be mixed and matched as applications and consumers' needs dictate. Hydra's first and primary render delegate is the rasterizing **Storm** renderer, which began as a modern OpenGL renderer, and which has now incorporated a “graphics interface” abstraction that allows Storm to use Vulkan, Metal, and potentially other rasterizing rendering API's. Storm is highly scalable, multi-pass, and uses **OpenSubdiv** for mesh rendering. The repository also includes with a simple **Embree** -based path tracer to serve as an example for creating more backends, and HdPrman, which is evolving into the definitive means of rendering USD with **Pixar's premiere RenderMan renderer**.

The USD scene delegate to Hydra is used in **usdview** and nearly all third-party plugins that integrate USD, and is meant to provide a “ground truth” rendering of any scene composed of prims conforming to the **UsdGeom schemas**, UsdShade, UsdVol, UsdSkel, UsdLux, and other graphics-related schema domains. It also provides fast preview and animation streaming for USD scenes.

USD can be extended/customized:

Even though USD is primarily used as an embedded sub-system, the breadth of the problem-space it covers demands that it be extensible along a number of axes. USD comes with its own [plugin discovery mechanism](#), and the following plugin-points:

- **Asset Resolution**

In a highly-referenced scene, it can be advantageous to have a degree of separation between the asset paths recorded in the USD files and the “resolved locator/identifier” from which the asset will ultimately be loaded. The [ArResolver](#) interface can be customized per USD installation and per “plugin package”, allowing, for example, site-specific naming conventions to be resolved, and for dynamic versioning control to be applied. USD ships with a default resolver implementation that allows for simple “search-path” style asset resolution for traditional filesystems.

However, the Ar system allows for the coexistence of multiple, URI-protocol-dispatched resolvers, each of which can resolve asset paths to [ArAsset](#) that can stream data directly from clouds or databases, or even construct assets procedurally, in-memory.

- **File Formats**

A USD Layer can be taught to be populated with data translated from any kind of compatible file format, by implementing an [SdfFileFormat](#) plugin for the format. USD’s own native *usda* (text), *usdc* (binary), and *usdz* (packaged archive) formats are implemented this way, as is the included support for reading Alembic files via the [Alembic USD Plugin](#), as well as [MaterialX xml files](#).

File formats can also be “dynamic”, such that when referenced into a scene via a [payload arc](#), modifiable metadata parameters on the payloaded prim are transmitted to the file format plugin which is then allowed to re-evaluate itself. This allows for a degree of user-directed proceduralism.

- **Schemas**

USD includes [a tool for generating new schemas](#) (C++ classes, python bindings, and all required boilerplate) from a simple usda text description of the schema. This can be used to add new USD prim schema types and API’s to your pipeline or package, with which you will be able to interact in your application-level plugins just as if they were native USD schemas. For typed schemas that are conceptually imageable, you can also teach Hydra how to image them. You can additionally use [OpenExec](#) to register computations for schemas for better performance characteristics in addition to data invalidation and caching management.

What can't USD do?

No GUIDS

USD uses a textual, hierarchical namespace to identify its data, which means it is “namespace paths” by which overrides bind to their defining prims/properties. In consequence, when the internal namespace of a referenced asset changes, *higher-level overrides previously recorded in referencing assets will fall off*. One solution to this problem is to identify data by a “globally unique identifier” (GUID), and then associate overrides with the same GUID as the defining prim. While solving the namespace-editing problem, GUIDs introduce other problems into a pipeline, and potentially limitations on flexibility of composition. In past iterations of USD, Pixar used a form of GUID at the model/asset granularity, and after carefully weighing the pros and cons, we have decided that for us, the cost of occasional “namespace fix-up” operations run over a collection of assets is worth paying for the ease of asset construction and aggregation, and readable text asset representations that we get from namespace-paths as identifiers.

Not a rigging system

USD provides a lightweight, optimized scenegraph to facilitate authoring and efficient extraction of composed scene description. However, USD's scenegraph leans towards a “low-memory footprint, higher-latency data access” tradeoff, whereas a high-performance rigging system requires more of a “high-memory footprint, low-latency access to data” tradeoff.

The [OpenExec](#) computation engine does provide a general purpose computation framework that could be used to develop a rigging system, but OpenExec by itself is not a rigging system. See [What OpenExec Is Not](#) for more details.

Further, the more rigging behaviors and execution semantics we would add to USD, the more difficult it would become to interchange the data successfully between DCC's, since there is not, currently, broad agreement between vendors of what these behaviors should be.

USD and its schema generation tools should be suitable for encoding rigging for round-tripping rigging data in a particular application or custom pipeline, and USD does provide facilities that a client could use to build more extensive in-memory caches on top of a UsdStage to provide lower-latency access to data encoded in USD. But for now, these do not play a significant role in what we feel is the primary directive of USD: scalable interchange of geometric, shading, and lighting data between DCC's in a 3D content creation pipeline.

Heritage of USD at Pixar

USD is roughly the fourth generation of “composed scene description” developed at Pixar. After muscling through [Toy Story](#), in which each shot was described by a single, linear program file, the Pixar R&D team began adding and evolving concepts for referencing,

layering, editing, and variation in the context of its proprietary animation system, *Marionette* (known internally as *Menv*), beginning with *A Bug's Life*, and continuing over the course of the next ten feature films.

By 2004 it was clear that, although *Marionette* had grown quite powerful, its organically evolved provenance was becoming a hindrance to continued stable development and our ability to leverage important tools like multi-core systems. The studio committed to the design and development of a ground-up, second-generation animation system now known as *Presto*, which was first used on *Brave* and all features since. One of the problems with *Marionette* that *Presto* set out to address was that its various features for composing and overriding 3D scene description could not always be used together effectively, because they were spread across three different formats and “composition engines”. *Presto* delivered a second generation of scene description that was *unified*, enabling referencing, overriding, variation, and other operations at all granularities from a single mesh, to an entire model, to an environment or shot, encoded in a single text format and evaluated with a single composition engine.

However, at the same time, Pixar, along with much of the film and effects industry, found it advantageous to transition from a pipeline in which animation and rigging were kept live up until rendering, to one in which animation and rigs were baked out into efficient “pose caches” containing animated posed points and transforms, so that lighting, effects, and rendering could reduce the latency (and memory footprint) with which they can access the data. Consequently, in 2008-2009, the pipeline development team began building *TidScene*, a geometry schema backed by a binary database (Berkeley DB), with a lightweight scenegraph as the mechanism for authoring and reading time-sampled data. Key elements of *TidScene* included a (for the time) high performance OpenGL rendering plugin that enabled direct-from-*TidScene* preview rendering in all pipeline applications, and the development of a native referencing feature that was used (possibly abused) to achieve layering, scenegraph “isolation” (i.e. loading only a portion of the scene), asset referencing, and some support for variation.

The speed, scalability, and universal pipeline access of *TidScene* pose-caches were a success, but also put Pixar back into a place where we had multiple, competing systems for creating composed scene description, with different semantics, API's, and places in the pipeline where they could be used. The mandate for the USD project, initiated in 2012, was to marry the (recently redesigned and improved) composition engine and low-level data model from *Presto* with the lazy-access, time-sampled data model and lightweight scenegraph from *TidScene*. USD delivers an all-new scenegraph that sits on top of the very same composition engine that *Presto* uses, and has introduced parallel computation into all levels of the scene description and composition core.

A key component of the USD project was the development of a modern, scalable rendering architecture, dubbed *Hydra*, initially deployed with what would become known as the Storm high-performance rasterizing renderer. *Hydra* ships as part of the USD project because it adds tremendous value to USD adoption in a pipeline and is used in all our plugins; it also

provides a benchmark and reference for how to leverage USD's multithreading for fast scene loading and imaging, as well as updating efficiently in response to dynamic edits to a live UsdStage. However, Hydra is a product in its own right, and already has other direct front-end couplings other than USD (including Presto and Maya, Katana, and Houdini plugins), and has grown beyond its original OpenGL-inspired architecture to service other render delegates, such as path-tracers.